# Concentric and Archimedes Chess

Version 1
Group 20


| | | | |
|---|---|---|---|
| Josef Graf | grafj1@mcmaster.ca | 400443948 | Project Manager, Developer |
| Carter Naganobu | naganobc@mcmaster.ca | 400471871 | Developer |
| Roland Kim | kimr30@mcmaster.ca | 400407536 | Developer |
| Skyler Suk | suks2@mcmaster.ca | 400411488 | Developer |
| Harshmeet Singh | singh27@mcmaster.ca | 400313448 | Developer |
| Harkirat Singh | singh75@mcmaster.ca | 400434064 | Developer |
| Sparsh Poddar | poddas2@mcmaster.ca | 400409187 | Developer |



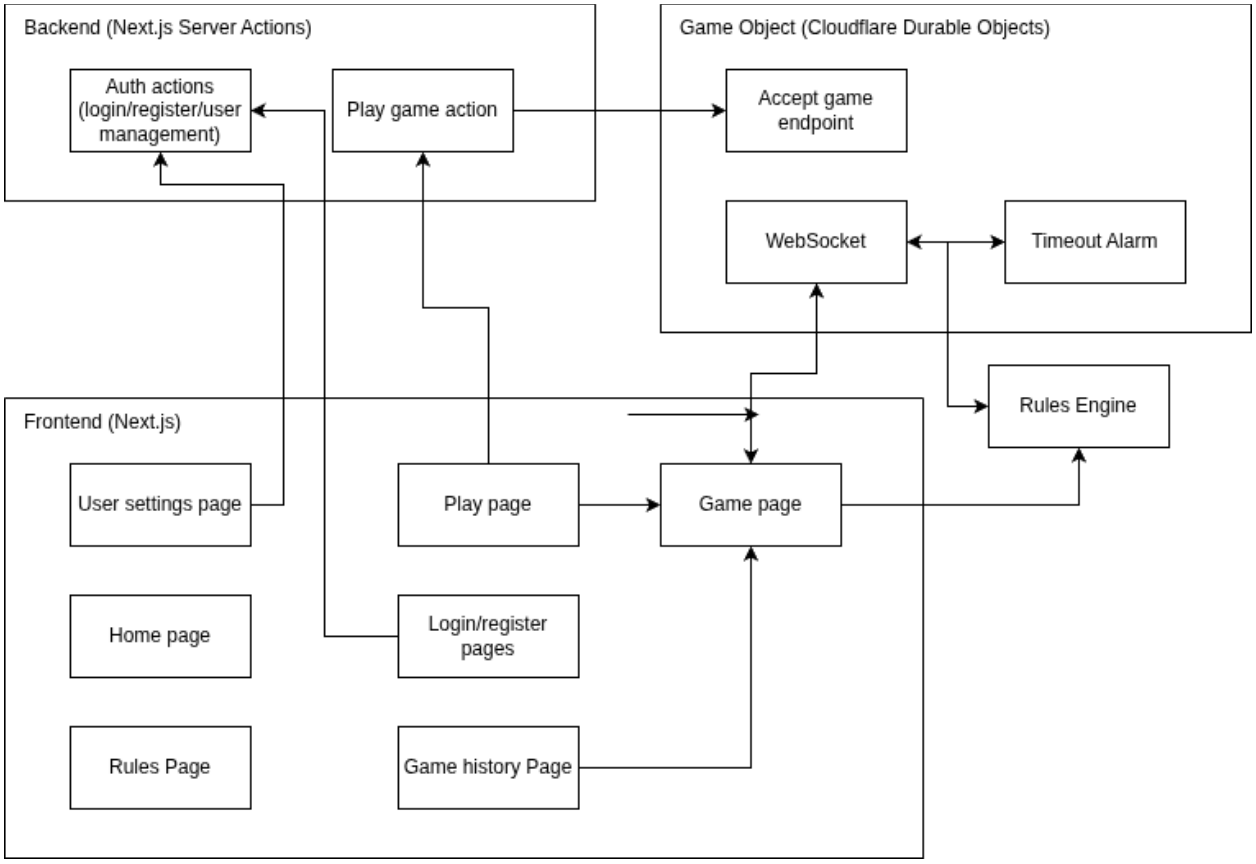
**Design Document ..... 3**
3. Brief ..... 3
4. Component Diagram ..... 3
5. Relationships ..... 3
Backend (Next.js Server Actions) ..... 3
Game Object (Cloudflare Durable Objects) ..... 4
Frontend (Next.js) ..... 4
6. Components ..... 5
Accept game endpoint ..... 5
Game WebSocket ..... 6
Timeout Alarm ..... 6
Board Renderer ..... 6
7. User Interface ..... 9
8. Verification & Validation (V&V) Plan ..... 10
8.1 Frontend V&V ..... 10
8.2 Backend V&V ..... 12
Appendix ..... 14
8. Verification & Validation (V&V) Plan (Frontend) ..... 14
9. Verification & Validation (V&V) Plan (Backend) ..... 18
Figma UI Renders ..... 23

# Design Document

## 3. Brief

The purpose of this project is to create a web-based, multiplayer implementation of the concentric and archimedean chess variants. In particular, the variants introduce new board shapes, that of concentric circles and an archimedean spiral respectively, as well as a new piece and changes to the way existing pieces move resulting in drastically different gameplay and strategies than the traditional chess game. This document is meant to provide an overview of the design of the user interface and the various components that make up the project.

## 4. Component Diagram




## 5. Relationships

### Backend (Next.js Server Actions)

All general backend tasks are handled by Next.js Server Actions, which allow for calling API endpoints on the server in-line in React code without having to manage effects or directly fetching endpoints.

- The user settings and login/register pages interact with the auth actions to perform all user management and authentication actions such as logging in and registering (which returns a session cookie) and changing names and emails.
    - Requirements 6.2.1, 6.2.2, 6.3.4
- The play page first uses the play action which checks for any players currently searching for a game and, if found, uses the accept game endpoint on the associated Game Object to accept the game. If not, it creates a new game in the database. In either case, it redirects the user to the game page for the new page.
    - Requirements P1.1, P2.1, 6.3.4
- The repository code for the backend follows proper TypeScript and React best practices, with inline documentation for each API endpoint and major component.
    - Requirements 6.5.1, 6.5.2, 6.5.3

**Game Object (Cloudflare Durable Objects)**

All backend tasks specific to a particular game are handled by Cloudflare Durable Objects that ensures that only a single instance of the edge runtime is running for every specific object (referenced by ID). All code running for a particular game can be made atomic to ensure that race conditions cannot occur but concurrent execution of requests for different games can still take place.

- The play action calls the accept game endpoint to atomically accept the game, ensuring that no other players trying to accept corrupts the database.
    - Requirements P2.1
- The game page opens a connection to the game WebSocket and listens for messages for moves, resignations, draws, etc. in the game and sends events when the player makes a move, resigns, draws, etc.
    - Requirements P0.1, P0.3, P0.4, P0.5, P1.2, 6.1.1, 6.1.2, 6.1.3, 6.1.5, 6.2.3, 6.3.4, 6.3.1, 6.3.2, 6.3.3
- The game WebSocket makes use of Durable Object Alarms, scheduling a callback to run when a player's time runs out, prompting the game object to mark a game finished in the database and send messages out to all listening clients.
    - Requirements P0.5
- The game WebSocket also makes use of the shared rules engine code to determine if a move is valid when a player attempts to make one.
    - Requirements P0.1, P0.2, 6.1.4
- The game WebSocket stores both games and moves in tables that are rendered in the frontend history pages to be read back and displayed in order to recreate previous board states and piece positions.
    - Requirements P1.5, 6.2.4, 6.3.2
- The game IDs map to individual Durable Object instances, which gives strong game consistency across many different requests from concurrent players at about low to moderate estimated levels of traffic.
    - Requirements 6.7.1, 6.7.2

**Frontend (Next.js)**

All frontend tasks for the general views and running games are handled by Next.js frontend, a full client-facing application that provides the UI and components for display,

navigation, and layouts. The UI components and toasts are rendered using Next.js, lucide-react, sonner, react-day-picker, and Tailwind CSS.

- The game page displays the current board state, showing the pieces as well as potential legal moves and confirmation/error prompts when a piece is moved. The page shows both usernames and ratings for the user UI. Game pages are available to spectators, who can see the live board state alongside the players. Navigation of the board and piece moves are accessible through key shortcuts.
    - Requirements P0.1, P0.3, P2.2, P3.1, P3.3, 6.3.3, 6.4.2, 6.4.3, 6.7.5
- The game page supports notifications made visible to the player when not actively playing, such as surrendering, offering a draw, or desynchronization.
    - Requirements P1.3, P1.4
- The history page displays replays of previous games between players, displayed similarly to a game page, including moves taken, names/ratings, and outcomes.
    - Requirements P2.3, P3.2
- The play page supports options for automatic matchmaking and invite links, which allow players to both connect to the Next.js server in order to initialize and start a variant chess game on a new game page.
    - Requirements P1.2, P2.1
- The login/register pages display relevant forms and fields for user authentication, and save the user data frontend to be displayed in the user's client pages.
    - Requirements P1.1
- The pages and UI have simple high-contrast ratio and colorblind buttons, indicators, and links for usability and accessibility, with a tutorial/rules page in order to properly instruct players to the rules of each variant chess game. Pages have compatibility with common browsers (Chrome/Firefox/Edge) and can adjust page layouts to different screen sizes (mobile, desktop).
    - Requirements 6.4.1, 6.4.4, 6.6.1, 6.6.2, 6.6.3, 6.7.3, 6.7.4

# 6. Components

## Accept game endpoint

- Normal behavior: should accept an open game, or deny acceptance of a full game.
- API: POST request to /do/games/:gameId/accept with session_token cookie set in the Cookie header and the body.
    - Status 200 -> accepted.
    - Status 409 -> already claimed.
- Implementation: no classes or methods used, just a series of cases to determine if they are already part of the game (accepting an invitation) and to figure out whether they are white/black, then update the database and broadcast to all clients.
- Potential undesired behavior: database error, broadcast to clients fails (or message is not delivered due to network issue).

## Game WebSocket

The game WebSocket is implemented as a series of generic parsing steps to fetch the message body and determine the message type, then a case statement is used to determine which type of message is being processed:

- Messages with type “move” represent a player attempting to make a move. It expects a JSON data field with the values side (white or black), the “from” square, and the “to” square of the move. The WebSocket then checks to see if the player is allowed to move for the specified side, then generates all legal moves using the rules engine, checks if the move is legal, then updates the database and all stored metadata with the new move, updated time remaining, etc. Any unexpected issues here are database errors, network errors, client formatting errors (bad request body), or logic errors.
- Messages with type “resign” represent a player resigning from the game. This message takes no data and simply updates the database with the resignation state and broadcasts it to all clients. Any unexpected errors here are database errors or network errors, client formatting errors should not be possible unless they happen before the specific message is handled.
- Messages with type “accept” represent a player accepting an invitation to a game. This message takes no data and simply updates the database with the acceptance state and broadcasts it to all clients. Any unexpected errors here are database errors or network errors, client formatting errors should not be possible unless they happen before the specific message is handled.
- Messages with type “sync_request” represent a client requesting a sync of game data to display. This message takes no data and simply returns the list of moves and current game state for the client to display. This occurs when the client is out of sync due to a network error or other issue. Any unexpected errors here are database errors or network errors, client formatting errors should not be possible unless they happen before the specific message is handled.

## Timeout Alarm

After every move is made, the current alarm is cleared and a new one is set for when the player’s time will run out. When that happens, Cloudflare calls the alarm handler which fetches game information, determines if time did run out or the alarm was not cleared previously, then updates the game state in the database to mark the loss and broadcasts the loss to all connected clients. Any unexpected errors here are database or network errors.

## Board Renderer

- Normal behavior: Should render the circular chess board by loading and parsing the SVG board file, computing cell coordinates, and displaying all pieces in their correct positions based on the current move history. Should allow users to select pieces, highlight the selected piece, display all legal move destinations, and submit a move when a legal destination is clicked. Should remain synchronized with game updates received from the server.

- API:Inputs:
    - moves: Move[] — move history used to reconstruct board state
    - onMove(from: string, to: string) — sends a move to the server via WebSocket
- Outputs:
    - Visual rendering of board and pieces
    - Invokes onMove when a move is made
- Implementation: Implemented as a React functional unit using hooks. On mount, fetches /cc/ccboard.svg and parses all SVG paths to extract cell identifiers and geometry. Cell center coordinates are computed using getBBox(). Piece positions are reconstructed from move history using the shared rules engine. User interaction is handled through click-based selection logic: selecting a piece computes legal moves, and selecting a legal destination updates local state and triggers the move callback. Rendering uses layered SVG for board cells and absolutely positioned HTML elements for pieces.
- Potential undesired behavior: SVG parsing may fail if the board asset is missing or malformed. Browser inconsistencies in getBBox() may cause incorrect positioning. Local board state may temporarily desynchronize if WebSocket messages are delayed or dropped. Rapid user interaction may trigger duplicate move attempts.

**Piece**

- Normal behavior:Should display a single chess piece at a specified board position using an optimized image. Should visually indicate selection and forward click events to the board renderer.
- API:Inputs:
    - Piece metadata (type, color, position, pixel coordinates)
    - Optional click handler and selection state
- Outputs:
    - Rendered piece image with interaction handling
- Implementation:Uses Next.js Image for optimized loading. Pieces are positioned with absolute CSS positioning and centered via transforms. Selection applies scale and drop-shadow effects. Click events are intercepted and propagated upward. Browser drag behavior is disabled.
- Potential undesired behavior: Missing or inaccessible image assets may cause invisible pieces. Incorrect layering or z-index values may render pieces beneath the board. Pointer event misconfiguration may prevent selection.

**Game View**

- Normal behavior: Should manage an active game session by maintaining client-side game state, coordinating WebSocket communication, and rendering the board and surrounding UI. Should handle the full game lifecycle including invitation acceptance, active play, and game termination.
- API: Inputs:
    - Initial game metadata and move history

    - Current player identifier (if authenticated)
- Outputs:
    - Sends WebSocket messages (move, accept, resign)
    - Renders game state, player info, and status indicators
- Implementation: Implemented as a React view that owns the authoritative frontend game state. On mount, establishes a WebSocket connection through the socket manager and registers a message listener. Incoming messages update local state using a case-based dispatcher. The view conditionally renders UI states such as waiting for acceptance, active gameplay, and game over screens. Cleanup logic disconnects the WebSocket and unregisters listeners on unmount.
- Potential undesired behavior:WebSocket connection failures may prevent gameplay. Out-of-order or missing messages may desynchronize the UI. Improper cleanup may leave orphaned connections. Latency may cause visible delays between move submission and confirmation.

**Landing Page**

- Normal behavior: Should introduce the project, highlight key features and variants, and provide navigation to gameplay and documentation.
- Implementation: Implemented as a server-rendered Next.js view styled with Tailwind CSS. Uses responsive layouts and optimized images to present feature highlights and calls to action.
- Potential undesired behavior: Slow image loading may cause layout shifts. Broken routes may lead to dead navigation links. Visual styles may degrade on older browsers.
-

**Rules Overlay**

- Normal behavior:
  Should present the rules panel inside a modal overlay that can be opened and dismissed by the user.
- API Inputs:
    - isOpen: boolean
    - onClose() callback
- Outputs:
    - Invokes onClose when dismissed
- Implementation: Conditionally renders a full-viewport overlay with a centered, scrollable dialog. Clicking outside the dialog or the close button dismisses the overlay. Internal clicks are prevented from bubbling.
- Potential undesired behavior: Background scrolling may not be fully prevented. Keyboard focus may escape the dialog. Accessibility support may be incomplete without ARIA metadata.

**Game History Page**

- Normal behavior: Should present previous games in a separate page and complete format, listing moves, players, and outcomes.
- API Inputs: Past game metadata and move history
- API Outputs: Renders game states, moves, player information, outcomes
- Implementation: A view page rendered by server Next.js and React styled with Tailwind CSS. The view renders the UI of different past game states under conditions such as the current move being viewed and player side selected.
- Potential undesired behavior: Failures in the network may display corrupted data in game history, which can require manual intervention.

## 7. User Interface

Our design philosophy was to focus on clarity by keeping things simple and clean. Allowing for the users to focus more on the chess game itself rather than clutter or unnecessary features.

Throughout the project's design, we commonly used a light/dark theme with orange accents for the base UI elements. The basic “grey” shades help to organize the webpage without distracting the user and the orange highlights generate attention to important content while being colour-blindness friendly.

Additionally, for the game boards, we use various shades of blue, yellow, green, and pink to highlight the current move’s relevant information to the player such as selected squares, legal moves, and/or move confirmation.

For the font choice, we chose Geist as it is a clean, legible font, with a monospace option that we use for specific scenarios with updating text such as the move timers.

We decided to have a common navigation sidebar on the pages of the website to help with feature discoverability and content navigation. From the sidebar, users can access the home page, game selection, and rules page, as well as either signup/login or settings and game history depending on if an account is logged in or not.

Parts of the design were inspired by an existing online chess application: chess.com. And a draft of the designs can be found on our Figma or in the Appendix.

The section below goes into more detail behind the purpose of each page and the important navigation that can be made. (ignoring navigation provided by the sidebar)

| | |
|---|---|
| Home Page | The purpose of the home page is to provide users with a brief introduction to the project as well as providing initial instructions on where to get started. From the home page, users will be able to move to account creation/login and game selection directly. |
| Sign Up/Log | The Sign Up/Log In pages are for users to input the appropriate fields and handle account creation and login. Once a user has successfully |

| | |
|---|---|
| In | completed their task, they will be sent straight to the home page, and the navigation sidebar will update. |
| Settings | The settings page is an area for users to modify their account information or log out. Logging out will send the user back to the homepage with an update navigation sidebar. |
| Game Selection | The game selection page is where users are able to select the gamemode and gametype they'd like to play. Once the game parameters have been confirmed, they will be sent to their game. |
| Game | The game itself consists of a background image of the board, an SVG layer for the intractable squares of the board, and the piece images. By having each square of the board be a custom SVG element, we can easily access and gather input on the squares of interest for highlighting or game logic. Additionally, the pieces are positioned overtop the board by referencing a specific CSS class to provide the correct position for the square. Beside the game board is a sidebar which shows relevant information about the game and provides meta-game options like draw or surrender. |
| History | The history page is for users to see their past games' results. From there, they can select a past game and work with a replay loaded in a Game page. |
| Rules | The rules section is an overlay which has all the rules of the game and can be accessed from various points in the site. |

# 8. Verification & Validation (V&V) Plan

Below is a condensed format of our Verification and Validation plan, the uncondensed version can be seen in the appendix.

## 8.1 Frontend V&V

Scope: Ensure UI correctness, real-time synchronization, and responsiveness.

Component Testing:

| Component | Unit Tests | Undesired Behaviors | Performance Metric & Methodology | Implementation Status |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| Navigation & Auth | Verify route protection (protected routes redirect unauthenticated users); confirm nav state updates after login/logout; validate correct page routing | Stale nav state; unauthorized access | Route transition latency: Measure client-side time from user click to first content render. Target: <200 ms avg. Benchmark: 50+ navigation events. | Implemented |
| Lobby & Matchmaking | Validate only valid configs submit; ensure loading states prevent duplicate requests; confirm redirect to game after match | Duplicate requests; UI fails on timeout; incorrect params sent | Matchmaking → game render: Measure time from matchmaking confirmation to game page fully loaded (excluding backend time). Target: <500 ms. Benchmark: 20+ matchmaking sequences. | Implemented |
| Live Game Board | Validate piece selection & legal move highlighting; ensure illegal moves rejected; verify board state updates on move events; test reconnection after simulated network loss | Board desynchronization; moves out of order; wrong turn indicator; accepting input when not player's turn | WebSocket latency: Measure time from message receipt to board visual update. Target: <100 ms. Methodology: inject timestamped move events; measure p50/p95 over 100+ moves per game. | Implemented (except for reconnection test, although manual tests show it works) |

| | | | | |
|---|---|---|---|---|
| Timers & Feedback | Verify timer decrements accurately; confirm highlights match state; ensure timeout/resignation display immediately; validate error banners appear | Timer drift; persistent highlights; missing visual feedback | Timer drift: Measure jitter over extended sessions. Target: <50 ms drift per minute. Methodology: compare UI timer vs server-side elapsed time over 1+ hour sessions. | Implemented (no error banners in the final implementation so tests not implemented) |
| Account & Profile | Validate form constraints & error messages; confirm updates reflect in UI; verify history renders correctly; ensure logout clears session state | Cached data after logout; misleading errors; inconsistent history | Page load latency: Measure initial content render after navigation. Target: <300 ms. Benchmark: 20+ page navigations. | Implemented |

Validation Outcome: All unit tests pass across supported browsers; real-time gameplay synchronized with backend under normal conditions; all performance metrics ≥targets; no critical UI desync during extended play.

## 8.2 Backend V&V

Scope: Ensure server-authoritative gameplay, deterministic rules engine, and performance targets.

Component Testing:

| Component | Unit Tests | Undesired Behaviors | Performance Metric & Methodology | Implementation Status |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| Rules Engine | Verify each piece's movement rules match spec (king leap, prince, bishop swing/twin, Berolina pawn, Archimedes exceptions); confirm board topology interpretation (neighbors, crescents/half-moons); test determinism (identical input = identical legal moves) | Legal move missing (false negative); illegal move allowed (false positive); rules regressions | Legal move calculation time: Measure rules engine entry/exit timestamps. Target: <100 ms per position. Methodology: benchmark over curated "hard positions" (many pieces + swing/twin interactions); report p50/p95 over 50+ positions. | Implemented |
| Game Accept Endpoint | Test atomicity: two simultaneous accepts → exactly one succeeds, one conflicts; verify already-in-game accept results in stable state; test DB write → broadcast success chain | Race condition corrupting ownership; DB error causing partial state; failed broadcast | Accept game latency (p95): Measure server-side duration + client-observed latency. Target: <150 ms typical move latency (accept is on critical path). Benchmark: 20+ parallel accept attempts. | Implemented |

| | | | | |
|---|---|---|---|---|
| Move WebSocket Handler | Test message schema validation (reject malformed); verify authorization (can't move for opponent, spectator can't move); test idempotency (duplicate move not double-applied); verify state atomicity (legal move = DB update + broadcast; illegal move = reject only) | Client format errors causing crashes; duplicate moves applied; illegal moves accepted; network errors causing desync | Move latency (p95): Measure client send → server apply → opponent receives. Target: <150 ms. Methodology: timestamp server "move accepted" broadcast; measure RTT distribution in load runs with 100+ concurrent games. Report p50/p95. | Implemented |
| Timeout Alarm | Test alarm scheduling resets after each move; verify correct winner on timeout persisted; detect & ignore stale alarms | Stale alarm ending game incorrectly; DB/network errors preventing persistence; timeout not broadcast | Timeout enforcement delay (p95): Measure "alarm fired" → "game marked finished + broadcast emitted". Target: <150 ms. Benchmark: 50+ timeout scenarios. | Implemented |
| Persistence & Replay | Test atomic move append (exactly one per accepted move); verify replay reconstruction (loaded move list replayed = same board state as live game); test result immutability after game termination | Missing/duplicated moves; partial writes desynchronizing history vs state; replay corruption | Move-write & query latency (p50/p95): Instrument DB calls. Target: <150 ms move latency (includes persistence). Benchmark: measured under simulated | Implemented |

| | | | | |
|---|---|---|---|---|
| | | | concurrency of 100+ games. | |
| Desync & Sync | Force client to miss broadcast → client issues sync_request → verify server returns authoritative move list → client state matches server | Sync returns stale/partial data; sync fails under DB/network issues | Resync time (p95): Measure sync request → client receives authoritative state. Target: within move latency budget (<150 ms). Benchmark: 20+ forced-desync scenarios. | Implemented |
| Matchmaking | Test no double-pairing (two players enter queue, get one game); test no duplicate creation (rapid user clicks) | Race conditions causing orphaned games or double-pairing; unauthorized creation | Matchmaking & redirect latency (p95): Measure server-action duration & redirect completion. Target: <150 ms. Benchmark: measured under moderate concurrency (20+ simultaneous queue entries). | Implemented |

Validation Outcome: All backend unit/integration tests pass; rules engine 100% correctness (all legal moves allowed, zero illegal moves accepted); desync recovery reliably restores authoritative state; all performance metrics ≥targets under sustained load (≥100 concurrent games).

# Appendix

## 8. Verification & Validation (V&V) Plan (Frontend)

### 8.1 Scope and Goals

The purpose of frontend verification and validation is to ensure that the user interface behaves correctly, remains responsive under normal usage, and accurately reflects the underlying game state provided by the backend. As the frontend is the primary point of interaction for users, testing focuses on correctness of user actions, synchronization with real-time game updates, and perceived performance, including latency and responsiveness.

Front-end validation is conducted using a combination of unit tests for UI logic and state handling, integration tests for real-time interactions, and performance measurements related to rendering speed and update latency.

### 8.2 Frontend Modules and Test Strategy

The frontend is divided into logical interaction layers, each verified independently to ensure correctness and robustness.

#### 8.2.1 Navigation & Session Flow

**Description**

This module manages application routing, authentication-aware rendering, and navigation state.

**Verification (Unit Tests)**

- Verify navigation options update correctly based on authentication state.
- Ensure protected routes redirect unauthenticated users to the login page.
- Confirm navigation links route to the correct pages.
- Validate navigation state updates immediately after login or logout.

**Undesired Behaviors Tested**

- Stale navigation state after authentication changes.
- Unauthorized access to protected routes.
- Broken or incorrect navigation links.

**Performance Metrics**

- Client-side route transition latency measured from user click to first content render.
- **Target:** < 200 ms on average.

#### 8.2.2 Game Lobby & Matchmaking View

### Description

This module allows users to select game variants and initiate matchmaking.

### Verification (Unit Tests)

- Validate correct UI state while matchmaking is in progress.
- Ensure invalid configurations cannot be submitted.
- Confirm loading and disabled states prevent duplicate matchmaking requests.
- Verify correct redirection to the game view after matchmaking completes.

### Undesired Behaviors Tested

- Multiple matchmaking requests caused by rapid user input.
- UI failing to update when matchmaking fails or times out.
- Incorrect game parameters sent to the backend.

### Performance Metrics

- Time from matchmaking confirmation to game page render.
- **Target:** < 500 ms (excluding backend matchmaking time).

#### 8.2.3 Live Game View (Board & Interaction Layer)

### Description

This is the most critical frontend module, responsible for rendering the game board, handling player input, and synchronizing real-time updates via WebSocket communication.

### Verification (Unit & Integration Tests)

- Validate piece selection and legal move highlighting logic.
- Ensure illegal moves cannot be submitted.
- Verify board state updates correctly upon receiving move events.
- Confirm UI reflects game-ending states (checkmate, resignation, timeout).
- Test reconnection and state resynchronization after simulated network interruptions.

### Undesired Behaviors Tested

- Board desynchronization after connectivity loss.
- Moves displayed out of order.
- Incorrect player turn indicators.
- UI accepting input when it is not the player’s turn.

### Performance Metrics

- WebSocket message receipt to board render latency.
- **Target:** < 100 ms from message receipt to visual update.

#### 8.2.4 Real-Time Feedback & Visual Cues

### Description

This module manages timers, move confirmations, highlights, and status indicators.

### Verification (Unit Tests)

- Verify timers decrement accurately and pause appropriately.
- Confirm visual highlights match the current interaction state.
- Ensure timeout and resignation indicators display immediately.
- Validate error banners appear for invalid actions.

### Undesired Behaviors Tested

- Timer drift during extended gameplay.
- Persistent highlights after state changes.
- Missing or delayed visual feedback.

### Performance Metrics

- Timer update jitter measured over extended sessions.
- **Target:** Visible drift < 50 ms per minute.

#### 8.2.5 Account & Profile Interface

### Description

This module includes login, signup, settings, and game history pages.

### Verification (Unit Tests)

- Validate form input constraints and error messages.
- Ensure successful updates reflect immediately in the UI.
- Confirm game history renders correctly.
- Verify logout clears all client-side session state.

### Undesired Behaviors Tested

- Cached user data persisting after logout.
- Incorrect or misleading error messages.
- Inconsistent history rendering.

### Performance Metrics

- Time to render account-related pages after navigation.
- **Target:** < 300 ms for initial content render.

### 8.3 Frontend Performance Validation Summary

| Metric | Target |
|---|---|
| Client-side route transition | < 200 ms |
| Matchmaking → Game render | < 500 ms |
| WebSocket update → UI render | < 100 ms |
| Timer drift | < 50 ms / minute |
| Page initial render | < 300 ms |

### 8.4 Validation Outcome

The frontend is considered validated when:

- All defined unit and integration tests pass across supported browsers.
- Real-time gameplay remains synchronized with backend state under normal network conditions.
- All performance metrics meet or exceed defined targets.
- No critical UI desynchronization occurs during extended gameplay sessions.

## 9. Verification & Validation (V&V) Plan (Backend)

### 9.1 Scope and Goals

Backend verification and validation ensures that the server:

- Enforces server-authoritative gameplay (clients can request moves, but only the server can accept/apply them).
- Implements a deterministic rules engine (pure state → next state / illegal), enabling reproducible testing.
- Handles desync by providing an authoritative resync path.
- Meets performance targets, including <150ms move latency, <100ms legal-moves calculation, and supporting ≥100 concurrent games.

Backend validation is considered successful when:

- Illegal moves are always rejected and legal moves are always accepted (per the project's "100% passing rate" correctness goal).
- Game state remains consistent across both clients under normal network conditions.

## 9.2 Backend Modules and Test Strategy

#### 9.2.1 Rules Engine (Game Logic Core)

Description

The rules engine computes legal moves and validates/executes moves based on the variant and current game state. It must encode variant-specific mechanics (e.g., crescents/half-moons/twins, king initial leap, prince behavior, Berolina pawn behavior).

### Verification (Unit Tests — deterministic "pure function" tests)

Unit tests to confirm that:

- Board topology rules are correctly interpreted (neighbors across edge/vertex; crescent/half-moon/twin definitions).
- Each piece's movement rules match the spec, including special cases:
  - King initial leap constraints.
  - Prince movement + "not subject to check".
  - Bishop swing/twin behavior and one-time color-change constraint.
  - Berolina pawn behavior on crescents/half-moons + promotion constraints.
  - Archimedes special movement exceptions (bishop/queen vertical, l6↔A7 pawn).
- Determinism: identical input state always yields identical output (same legal move set, same next-state result).

### Undesired Behaviors Tested

- "Legal move missing" (false negative) or "illegal move allowed" (false positive).
- Rules regressions when rules are updated/extended (important because the rules may be changed slightly later).

### Performance Metrics

- Legal move calculation time (per position): Target <100ms.
  Methodology: instrument rules engine entry/exit timestamps; run benchmark suite over curated "hard positions" (many pieces + swing/twin interactions) and report p50/p95.

#### 9.2.2 Game Object Runtime (Durable Object)

This module is responsible for game-specific atomicity and concurrency safety across requests for the same game ID.

### A) Accept Game Endpoint

### Description

Accepts an open game or denies a full/claimed game; assigns sides and broadcasts updates.

### Verification (Integration Tests)

- "First accept wins": two users attempting to accept simultaneously → exactly one receives success, the other receives conflict.
- Accepting an invite when already part of game results in stable/expected state.
- Database write success → broadcast success; database failure → no partial accept state.

**Undesired Behaviors Tested**

- Race condition corrupting game ownership (the design explicitly uses the game object to avoid this).
- Database error or broadcast failure.

**Performance Metrics**

- "Accept game" endpoint latency (p95), since it's on the critical path to starting play.
  Methodology: measure server-side duration + client-observed duration; report p50/p95 over N parallel accept attempts.

**B) Game WebSocket**

**Description**
Handles realtime messages: move attempts, resignation, accept, and sync request, with rule validation and persistence.

**Verification (Integration + Protocol Tests)**

- Message parsing + schema validation:
  - Valid move messages require side/from/to and are rejected on malformed input.
- Authorization:
  - Player cannot move for the opposite side; spectator cannot move.
- Ordering/idempotency:
  - Duplicate move messages do not double-apply.
  - Out-of-turn moves always rejected.
- State update integrity:
  - On legal move: database updated + broadcast emitted.
  - On illegal move: reject + no state mutation.

**Undesired Behaviors Tested**

- Client formatting errors causing crashes or inconsistent handling.
- Network errors causing message loss and client desync.
- Logic errors leading to accepting illegal moves (violates P0 server-authoritative rule).

**Performance Metrics**

- Move latency (client send → server apply → opponent receives): Target <150ms under normal conditions.

Methodology: add timestamps to server “move accepted” broadcast; measure end-to-end RTT distribution (p50/p95) in load runs.

**C) Timeout Alarm (Clock Enforcement)**

**Description**
Schedules an alarm after each move; on fire, determines if time expired and updates game state and broadcasts result.

**Verification (Integration Tests)**

- Alarm scheduling resets after each move (old alarms cleared; only latest timer counts).
- Correct winner/loser computed on timeout and persisted.
- Alarm fires but detects it was stale → no incorrect timeout result.

**Undesired Behaviors Tested**

- Stale alarm incorrectly ending game.
- Database/network errors causing timeout to not persist or not broadcast.

**Performance Metrics**

- Timeout enforcement delay: “alarm fired” → “game marked finished + broadcast emitted”.
  Methodology: record timestamp at alarm handler entry and after persistence; report p95.

#### 9.2.3 Persistence Layer (Games + Moves + Replay)

**Description**
Stores games as move sequences and metadata so replays can be deterministically reconstructed.

**Verification (Integration Tests)**

- Move append is atomic: exactly one move added per accepted move event.
- Replay reconstruction:
  - Loading move list from DB and replaying yields the same board state as the live game path (determinism check).
- Result storage:
  - Game termination writes {result, reason} and remains immutable afterward.

**Undesired Behaviors Tested**

- Missing/duplicated moves causing invalid replays.
- Partial writes that desync move history vs current state.

**Performance Metrics**

- Move-write latency (server side) and query latency for replay fetch. Methodology: instrument DB calls; report p50/p95 under simulated concurrency.

#### 9.2.4 Desync Handling & Sync Requests

**Description**

When a client is out of sync, it can request server state and refresh to the last committed state.

**Verification (Integration Tests)**

- Force client to miss a broadcast → client issues sync_request → server returns authoritative move list/state → client matches server state.
- Sync response is always consistent with DB state.

**Undesired Behaviors Tested**

- Sync returns stale/partial data.
- Sync path fails under intermittent DB/network issues.

**Performance Metrics**

- Time to resync (sync request → client receives authoritative state). Methodology: measure end-to-end timing and report p95.

#### 9.2.5 Matchmaking / Game Creation (Next.js Server Actions)

**Description**

General backend tasks not tied to one game instance (e.g., searching/creating games, login/register flows) are handled by server actions; play flow either finds a waiting player or creates a new game and redirects.

**Verification (Integration Tests)**

- Matchmaking correctness:
  - Two players entering queue get paired into one game (no double-pairing).
- No duplicate game creation under double-submit (rapid user clicks).

**Undesired Behaviors Tested**

- Race conditions in matchmaking leading to orphaned games or duplicate pairings.
- Unauthorized creation/accept when not authenticated (security constraints).

**Performance Metrics**

- “Find match / create game” server time and redirect completion time. Methodology: measure server-action duration; report p95 under moderate concurrency.

### 9.3 Backend Performance Validation Summary (Targets)

- Legal moves calculation time: <100ms
- End-to-end move latency: <150ms
- Concurrency: ≥100 concurrent games without performance drop-off

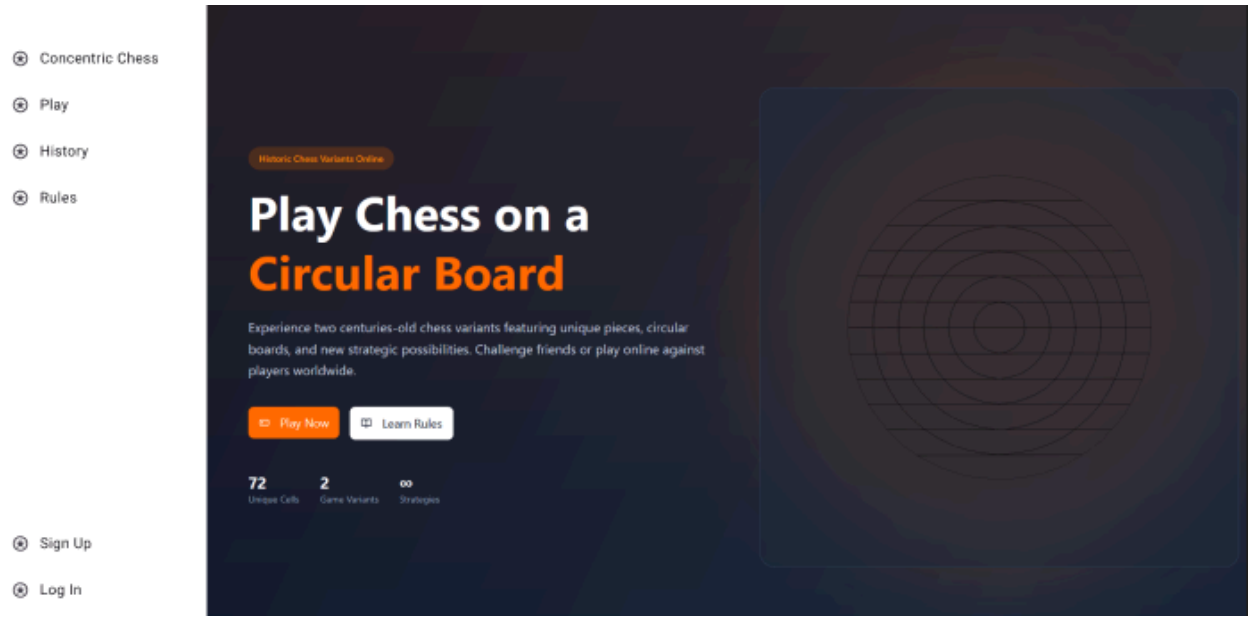
### 9.4 Validation Outcome (Backend)

Backend is considered validated when:

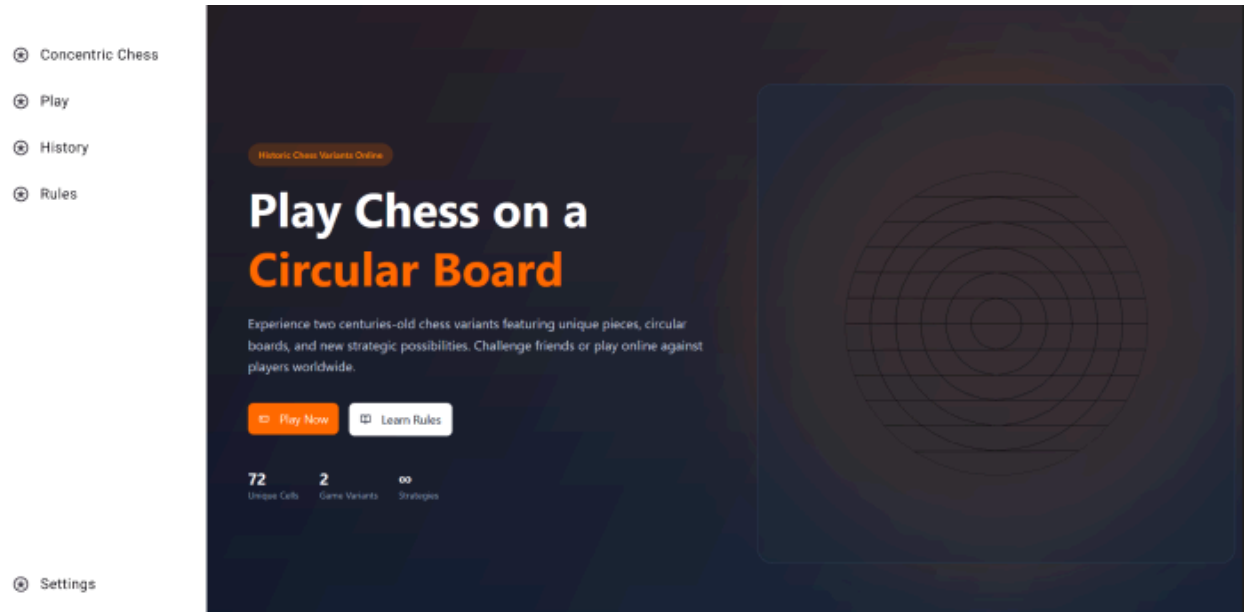
- All backend unit + integration suites pass.
- Rules engine correctness achieves the project goal that only legal moves can be made and all legal moves can be made (100% pass on the defined legality suites).
- Desync recovery reliably restores the authoritative state.
- Performance metrics meet targets across repeated runs and under simulated load.

## Figma UI Renders

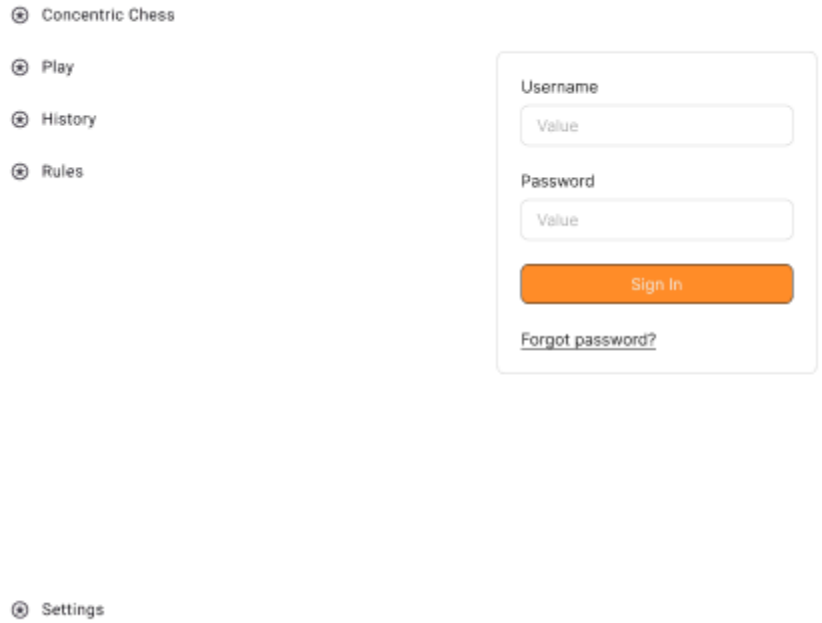
### Home Page (Not Signed In)



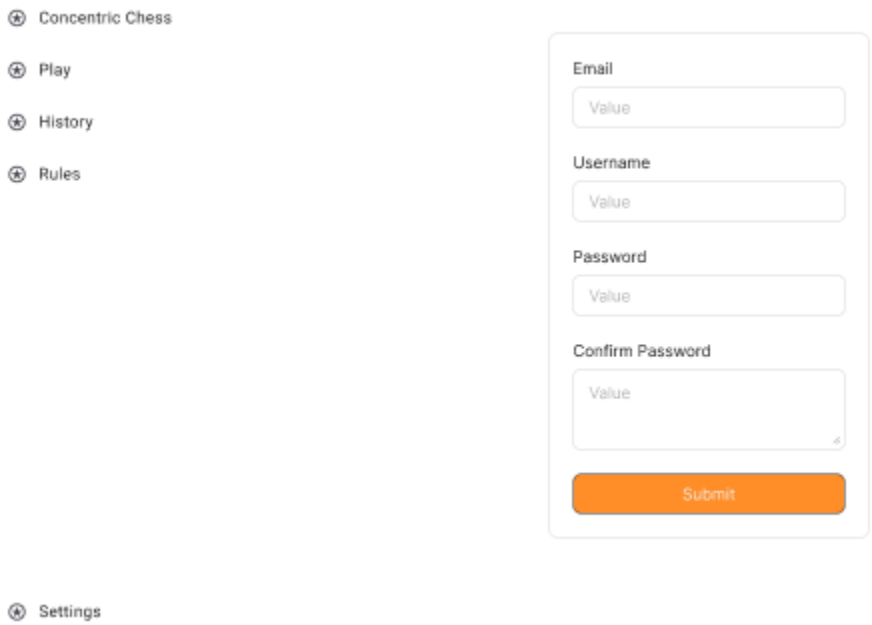
## Home Page (Signed In)



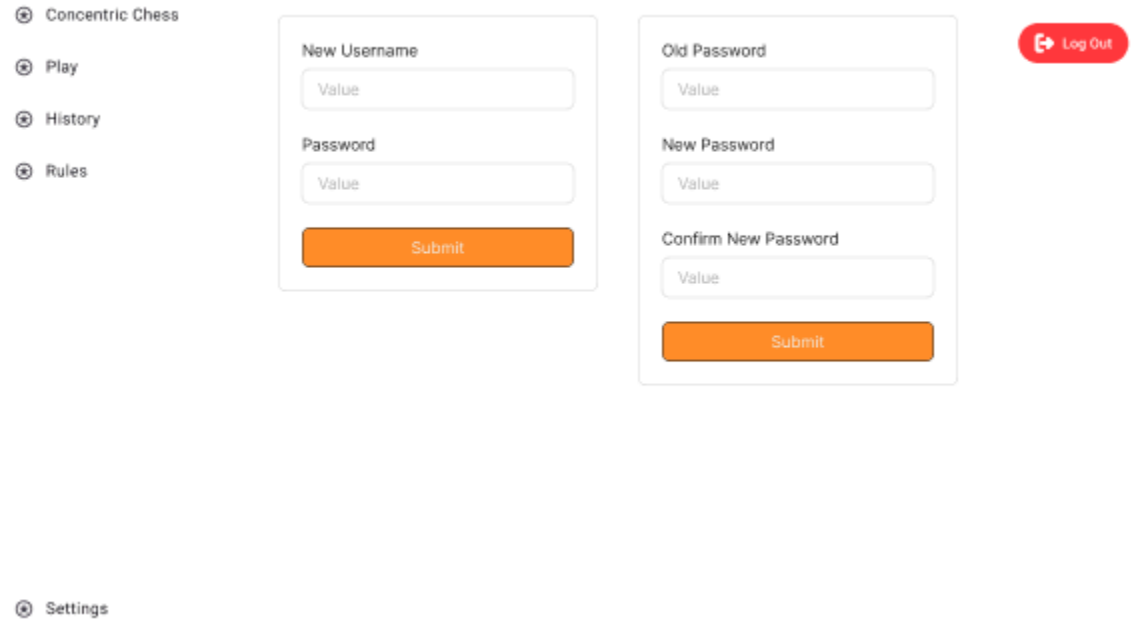

## Sign In



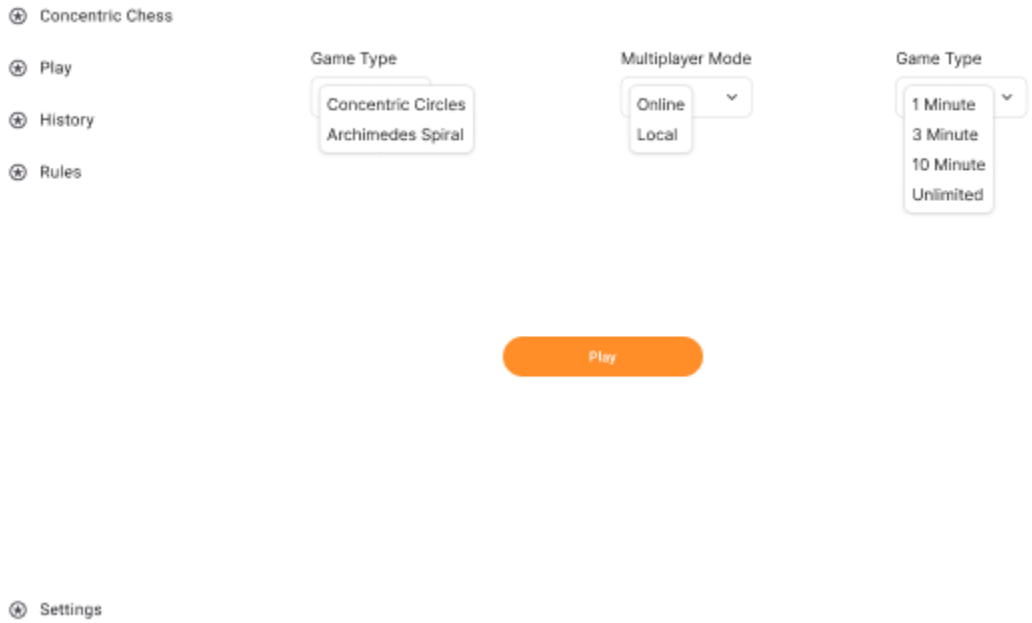
## Sign Up



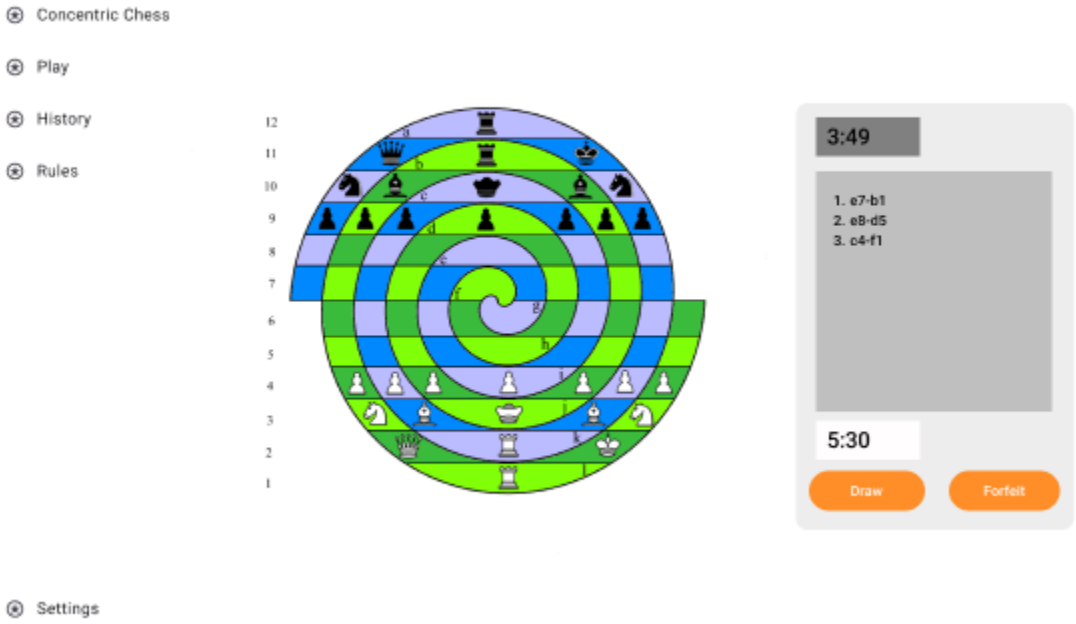

## Settings



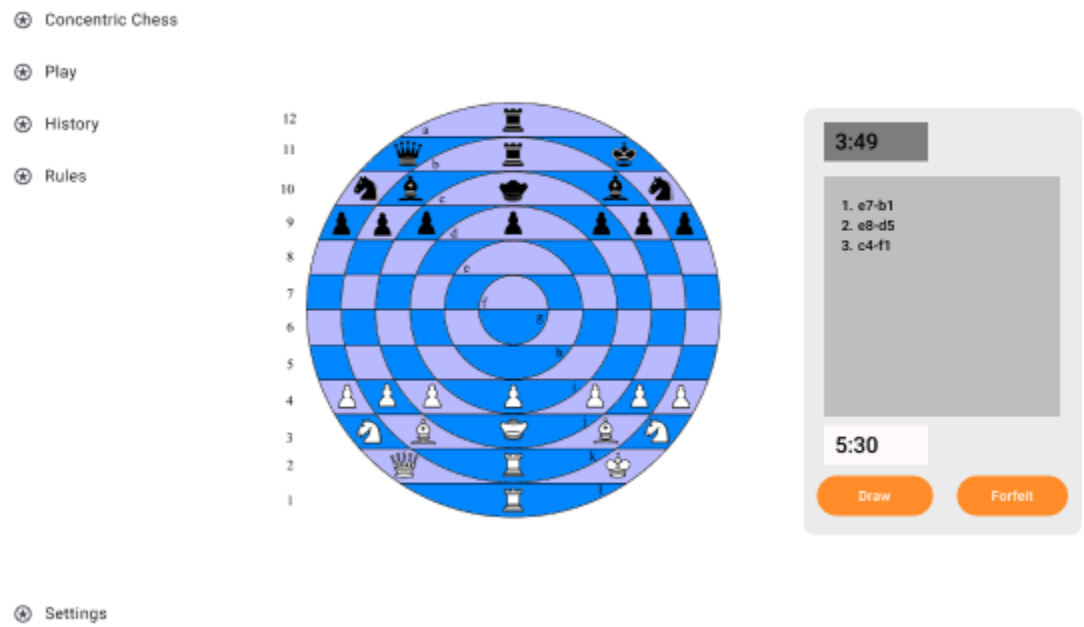
## Game Selection




## Archimedes Spiral Game



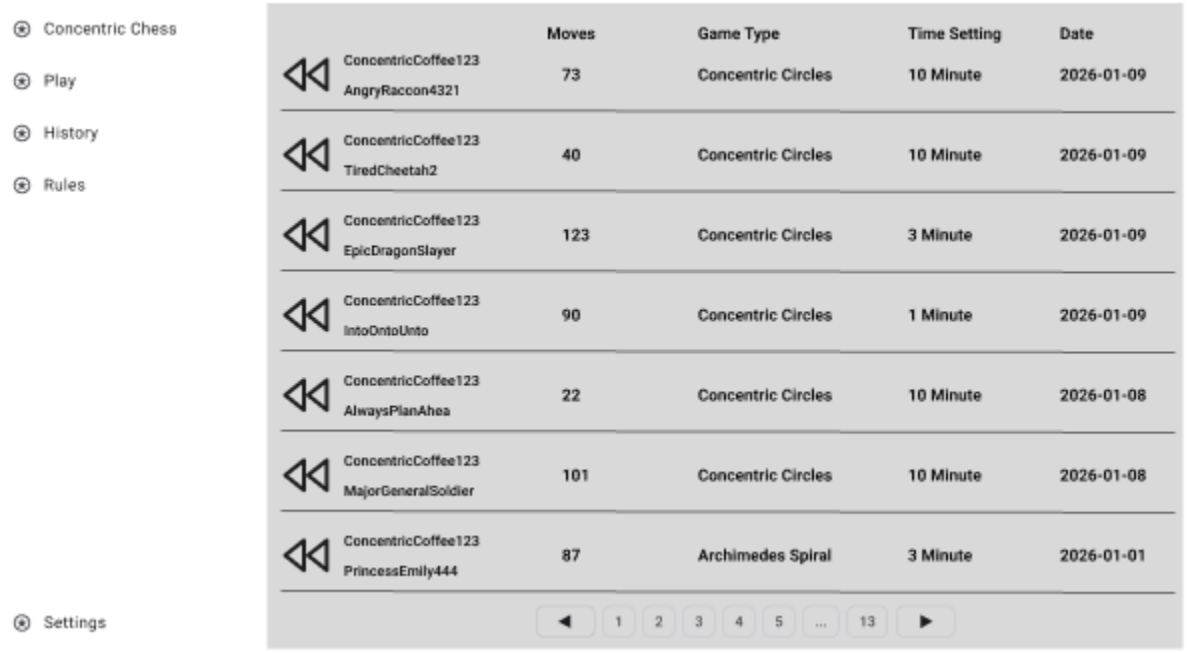
## Concentric Circles Game



## Rules



## History


Concentric Chess
Play
History
Rules
Settings
Moves
Game Type
Time Setting
Date
ConcentricCoffee123
AngryRaccon4321
73
Concentric Circles
10 Minute
2026-01-09
ConcentricCoffee123
TiredCheetah2
40
Concentric Circles
10 Minute
2026-01-09
ConcentricCoffee123
EpicDragonSlayer
123
Concentric Circles
3 Minute
2026-01-09
ConcentricCoffee123
IntoOntoUnto
90
Concentric Circles
1 Minute
2026-01-09
ConcentricCoffee123
AlwaysPlanAhea
22
Concentric Circles
10 Minute
2026-01-08
ConcentricCoffee123
MajorGeneralSoldier
101
Concentric Circles
10 Minute
2026-01-08
ConcentricCoffee123
PrincessEmily444
87
Archimedes Spiral
3 Minute
2026-01-01
◀
1
2
3
4
5
...
13
▶